

Manual de Instalación y Despliegue

Plataforma Bopacorp

Equipo de Infraestructura — Bopacorp

Versión	2.0 — revisada y ampliada
Fecha	Agosto de 2026
Sustituye a	Manual de Instalación y Despliegue, v1.0
Plataforma	Node.js 22, Docker Compose V2, Caddy 2
Servicios	bopacorp-api, bopacorp-crm, bopacorp-web
Duración estimada	60–90 minutos desde un servidor limpio

Índice

1	Introducción y Alcance	2
1.1	Alcance	2
1.2	Destinatario y conocimientos previos	3
1.3	Convenciones tipográficas	3
1.4	Cambios respecto a la versión 1.0	3
2	Arquitectura del Sistema	3
2.1	Visión general	3
2.2	Detalle de los servicios	3
3	Requisitos Previos	4
3.1	Sistema operativo	4
3.2	Recursos de hardware	4
3.3	Credenciales necesarias antes de empezar	4
4	Procedimiento de Instalación Paso a Paso	5
4.1	Paso 1 — Preparar el sistema base	5
4.2	Paso 2 — Instalar Git	6
4.3	Paso 3 — Instalar Docker Engine y Compose V2	6
4.4	Paso 4 — Instalar Caddy	7
4.5	Paso 5 — Obtener el token de GitHub Packages	7
4.6	Paso 6 — Obtener el proyecto de despliegue	8
4.7	Paso 7 — Configurar el archivo .env	8
4.8	Paso 8 — Ejecutar deploy.sh	8
4.9	Paso 9 — Verificar el despliegue local	9
4.10	Paso 10 — Configurar DNS	9
4.11	Paso 11 — Configurar Caddy	10
4.12	Paso 12 — Validación final sobre HTTPS	11
4.13	Paso 13 — Endurecer el servidor	11
5	Referencia de Variables de Entorno	11
5.1	Precedencia de VITE_API_URL	11

6	Configuración de Desarrollo Frente a Producción	12
6.1	Servidores de desarrollo en lugar de artefactos estáticos	13
6.2	Dockerfile.dev en la API	13
6.3	Volúmenes de código montados	13
6.4	Política de reinicio ausente	13
6.5	Recomendación	13
7	Operación y Mantenimiento	13
7.1	Control del ciclo de vida	13
7.2	Reconstrucción	13
7.3	Inspección	13
7.4	Gestión del espacio en disco	14
8	Actualización y Reversión	14
8.1	Actualizar el código de la aplicación	14
8.2	Actualizar la plataforma	14
8.3	Procedimiento de reversión	14
9	Seguridad y Endurecimiento	14
9.1	Cortafuegos	14
9.2	Docker y el cortafuegos: una precaución necesaria	15
9.3	Acceso SSH	15
9.4	Gestión de secretos	15
9.5	Actualizaciones automáticas de seguridad	15
10	Monitoreo y Observabilidad	15
10.1	Bitácoras	15
10.2	Recursos	15
10.3	Comprobaciones de salud	16
11	Solución de Problemas	16
11.1	Comandos de diagnóstico	16
12	Resumen Operativo	17
Anexo A: Plantilla Completa del Archivo .env		18
Anexo B: Documentación Oficial de Referencia		19

1 Introducción y Alcance

El presente manual documenta el despliegue de la plataforma Bopacorp, un sistema compuesto por tres servicios que se ejecutan como contenedores Docker sobre un único servidor anfitrión y que se exponen públicamente mediante nombres de dominio con HTTPS.

1.1 Alcance

El documento cubre la instalación desde un sistema operativo recién provisionado hasta la validación de los tres dominios públicos respondiendo sobre TLS, e incluye las tareas recurrentes de operación posteriores. Quedan *fuera* de alcance: el aprovisionamiento del servidor virtual en el proveedor de nube, la administración interna de Supabase y Cloudflare R2 como servicios gestionados, y el desarrollo o modificación del código fuente de los tres repositorios.

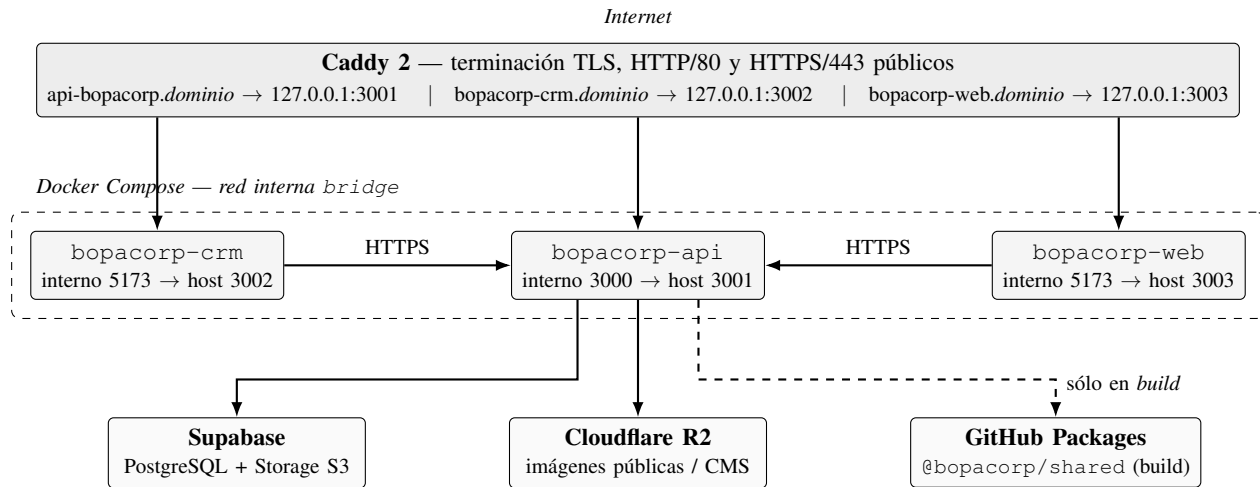


Fig. 1. Arquitectura de despliegue. Caddy es el único componente expuesto públicamente; los tres contenedores se enlazan a la interfaz de bucle local del anfitrión. La persistencia (base de datos, archivos e imágenes) es externa y está delegada en servicios gestionados.

1.2 Destinatario y conocimientos previos

Se asume que la persona que ejecuta el procedimiento posee acceso administrativo (`sudo`) al servidor, familiaridad básica con la línea de comandos de Linux y credenciales válidas en GitHub, Supabase y Cloudflare. No se requiere experiencia previa con Docker: todos los comandos necesarios se indican de forma literal.

1.3 Convenciones tipográficas

Los bloques monoespaciados con marco representan comandos que deben ejecutarse en el terminal del servidor o fragmentos de archivos de configuración. El símbolo ✓ indica un elemento obligatorio y ○ uno opcional o recomendado. Los marcadores `tudominio.com`, `<repo>` y `ghp_...` son valores de sustitución: deben reemplazarse por los valores reales del entorno antes de ejecutar el comando.

1.4 Cambios respecto a la versión 1.0

Esta revisión reordena la configuración de DNS para que preceda a la de Caddy —la emisión del certificado no puede completarse antes de que el nombre resuelva—, reestructura la instalación como una secuencia numerada con criterios de validación explícitos, e incorpora cuatro contenidos ausentes en la versión anterior: la distinción entre la configuración de desarrollo y la de producción (Sección 6), la rotación de bitácoras de Docker, un procedimiento de reversión y una plantilla completa de `.env` (Anexo A).

2 Arquitectura del Sistema

2.1 Visión general

La arquitectura, ilustrada en la Fig. 1, sigue el patrón de proxy inverso frontal. Caddy es el único proceso que escucha en los puertos públicos 80 y 443; recibe todas las peticiones entrantes, termina la conexión TLS y las reenvía por HTTP en la interfaz de bucle local al contenedor correspondiente según el nombre de host solicitado. Los tres contenedores no son alcanzables directamente desde Internet.

La persistencia no reside en el servidor. La base de datos relacional y el almacenamiento de currículums se delegan en Supabase, y las imágenes públicas del gestor de contenidos en Cloudflare R2. Esta decisión tiene una consecuencia operativa relevante: el servidor es, en la práctica, *desechable* y puede reconstruirse desde cero sin pérdida de datos, siempre que se conserven el archivo `.env` y el `Caddyfile`.

2.2 Detalle de los servicios

La Tabla I resume los tres componentes desplegables. Los frontends declaran `depends_on`: `api`, por lo que Docker Compose garantiza el orden de arranque, aunque no la disponibilidad efectiva de la API en el momento del arranque (véase la Sección 10 sobre *healthchecks*).

Tabla I
SERVICIOS QUE COMPONEN EL DESPLIEGUE

Servicio	Repositorio	Int.	Host	Dockerfile / base
api	bopacorp-api	3000	3001	Dockerfile.dev, node:22-alpine
crm	bopacorp-crm	5173	3002	Dockerfile, node:22-alpine
web	bopacorp-web	5173	3003	Dockerfile, node:22-alpine

Tabla II
SOPORTE POR SISTEMA OPERATIVO

Sistema	Soporte	Observaciones
Ubuntu Server 24.04 LTS	✓	Recomendado para producción
Ubuntu Server 22.04 LTS	✓	LTS estable, plenamente soportado
Debian 12 (Bookworm)	✓	Alternativa ligera equivalente
Rocky / Alma Linux 9	✓	Sustituir apt por dnf
Debian 11 (Bullseye)	!	Funcional; no recomendado para despliegues nuevos
Kali Linux	!	Sólo pruebas locales; nunca en producción
macOS / Windows	!	Sólo desarrollo local con Docker Desktop

Tres particularidades condicionan todo el procedimiento posterior:

- 1) El paquete bopacorp-shared **no se clona**. Es una dependencia npm privada que se resuelve desde GitHub Packages durante la construcción de las imágenes, lo que hace que la variable NPM_TOKEN sea un requisito *de compilación*, no de ejecución. Sin ella, la construcción falla antes de generar ninguna imagen.
- 2) La API se construye con Dockerfile.dev y monta ./bopacorp-api/src como volumen, habilitando recarga en caliente. Los dos frontends montan igualmente sus carpetas src y ejecutan el servidor de desarrollo de Vite.
- 3) Las variables con prefijo VITE_ se incrustan en el paquete JavaScript *en tiempo de construcción*. Modificarlas en .env no surte efecto hasta reconstruir la imagen.

El segundo punto es la desviación más importante respecto de una configuración de producción convencional y se analiza en la Sección 6.

3 Requisitos Previos

3.1 Sistema operativo

La Tabla II recoge el nivel de soporte por distribución. Para producción se recomienda Ubuntu Server 24.04 LTS con el núcleo actualizado, por su compatibilidad con los repositorios oficiales de Docker y Caddy y por su ciclo de soporte extendido.

3.2 Recursos de hardware

El dimensionamiento (Tabla III) está determinado por la fase de construcción, no por la de ejecución en régimen permanente. Compilar TypeScript y empaquetar con Vite en tres contenedores simultáneos es la operación de mayor consumo de memoria de todo el ciclo.

3.3 Credenciales necesarias antes de empezar

Conviene reunir los siguientes elementos antes de iniciar el Paso 1; su ausencia detiene el procedimiento a mitad de camino:

- ✓ Acceso sudo al servidor y su dirección IP pública.
- ✓ Cuenta de GitHub con acceso de lectura al repositorio Bopacorp/bopacorp-shared.
- ✓ Credenciales del proyecto Supabase (cadenas de conexión y claves S3).

Tabla III
DIMENSIONAMIENTO DEL SERVIDOR

Recurso	Mín.	Rec.	Justificación
CPU	2 vCPU	4 vCPU	Las construcciones de Node, TypeScript y Vite son intensivas en CPU y los tres contenedores compiten por núcleos
RAM	4 GB	8 GB	Los tres contenedores en reposo consumen 2,5–3 GB; cada <code>npm ci</code> más <code>tsc/Vite</code> añade 1–2 GB temporales
Disco	10 GB	20 GB	Imágenes Docker (1,5–2 GB), tres clones con <code>node_modules</code> (2–3 GB) y caché de construcción
Red	10 Mbps	100 Mbps	Clonado de repositorios, <code>npm ci</code> y descarga de imágenes base
Swap	2 GB	4 GB	Imprescindible con 4 GB de RAM o menos

Tabla IV
FLUJO COMPLETO DE INSTALACIÓN

#	Paso	Duración	Repetible
1	Preparar el sistema base	5 min	No
2	Instalar Git	1 min	No
3	Instalar Docker Engine y Compose V2	5 min	No
4	Instalar Caddy	3 min	No
5	Obtener el token de GitHub Packages	5 min	Al rotar
6	Obtener el proyecto de despliegue	2 min	No
7	Configurar el archivo <code>.env</code>	15–30 min	Al cambiar
8	Ejecutar <code>deploy.sh</code>	5–15 min	Sí
9	Verificar el despliegue local	3 min	Sí
10	Configurar DNS	5 min + propagación	No
11	Configurar Caddy	5 min	Al cambiar
12	Validación final sobre HTTPS	2 min	Sí
13	Endurecer el servidor	10 min	No

- ✓ Credenciales de Cloudflare R2 y nombre del *bucket*.
- ✓ Control sobre la zona DNS del dominio que se utilizará.
- ○ Dirección de correo para las notificaciones de expiración de certificados de Let's Encrypt.

4 Procedimiento de Instalación Paso a Paso

El procedimiento se compone de trece pasos que deben ejecutarse en el orden indicado. La Tabla IV ofrece la vista completa del flujo con una estimación de duración; los pasos 1 a 4 se ejecutan una única vez por servidor, mientras que los pasos 8 a 9 se repiten en cada actualización posterior.

4.1 Paso 1 — Preparar el sistema base

Objetivo: Actualizar los paquetes del sistema, instalar las utilidades necesarias para añadir repositorios firmados y garantizar que existe espacio de intercambio suficiente.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg lsb-release
```

Antes de continuar conviene comprobar los recursos disponibles:

```
lscpu | grep -E 'Model name|^CPU\(s\)'
free -h
df -h /
swapon --show
```

Si `swapon -show` no devuelve ninguna línea, no existe espacio de intercambio y debe crearse. En un servidor de 4 GB de RAM esta omisión es la causa más frecuente de que la construcción sea interrumpida por el gestor de memoria del núcleo:

```
sudo fallocate -l 4G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

# Reducir la tendencia a usar swap con RAM disponible
echo 'vm.swappiness=10' | sudo tee /etc/sysctl.d/99-swap.conf
sudo sysctl --system
```

Resultado esperado. `free -h` muestra una fila Swap con al menos 2 GB y `df -h /` indica un mínimo de 10 GB libres.

Si falla. Si `fallocate` no funciona sobre el sistema de archivos (algunos entornos con ZFS o contenedores), sustitúyalo por `sudo dd if=/dev/zero of=/swapfile bs=1M count=4096`.

4.2 Paso 2 — Instalar Git

Objetivo: Disponer de Git, que `deploy.sh` utiliza para clonar y actualizar los tres repositorios.

```
sudo apt install -y git
git --version

git config --global user.name "Nombre Apellido"
git config --global user.email "correo@dominio.com"
```

Resultado esperado. `git -version` devuelve una versión 2.x o superior.

Nota. La identidad de Git no es estrictamente necesaria para clonar, pero evita advertencias y resulta indispensable si en algún momento se crean confirmaciones desde el servidor.

4.3 Paso 3 — Instalar Docker Engine y Compose V2

Objetivo: Instalar el motor de contenedores y el complemento Compose V2, y habilitar su uso sin `sudo`.

La forma más rápida es el instalador oficial de conveniencia:

```
curl -fsSL https://get.docker.com | sudo sh
```

Si la política del entorno desaconseja ejecutar guiones remotos, la alternativa es añadir el repositorio manualmente:

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

A continuación se habilitan los servicios en el arranque y se añade el usuario al grupo `docker`:

```
sudo systemctl enable --now docker
sudo systemctl enable containerd

sudo usermod -aG docker $USER
newgrp docker # aplica el grupo en la sesion actual
```

Conviene además configurar desde el principio la rotación de bitácoras del demonio. Sin ella, el controlador `json-file` crece de forma ilimitada y termina agotando el disco tras varias semanas de operación:

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

```
sudo systemctl restart docker
```

Resultado esperado. `docker -version` devuelve 24.x o superior, `docker compose version` devuelve v2.x, y `docker run -rm hello-world` se ejecuta sin anteponer `sudo`.

Si falla. Si aparece `permission denied` al hablar con el `socket` de Docker, la pertenencia al grupo no se ha aplicado: cierre la sesión SSH y vuelva a entrar. Si `docker compose` no se reconoce como subcomando, falta el paquete `docker-compose-plugin`.

Advertencia. Pertenecer al grupo `docker` equivale funcionalmente a disponer de privilegios de superusuario en el anfitrión. Concédalo únicamente a las cuentas que administran el despliegue.

4.4 Paso 4 — Instalar Caddy

Objetivo: Instalar el proxy inverso que publicará los tres servicios con HTTPS.

Caddy no es opcional en este despliegue. Las variables `CORS_ORIGIN`, `API_PUBLIC_URL` y `VITE_API_URL` se configuran con nombres de dominio, de modo que el acceso por `IP:puerto` produce errores de origen cruzado en los frontends. La instalación no se considera terminada hasta que Caddy sirve los tres dominios.

```
sudo apt install -y debian-keyring debian-archive-keyring \
  apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
  | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update
sudo apt install -y caddy
sudo systemctl enable --now caddy
```

Resultado esperado. `caddy version` devuelve v2.x y `systemctl status caddy` muestra el servicio activo. Al visitar la IP pública en un navegador debe aparecer la página por defecto de Caddy.

Nota. La configuración del `Caddyfile` se realiza más adelante, en el Paso 11, una vez que los contenedores estén levantados y el DNS resuelva correctamente.

4.5 Paso 5 — Obtener el token de GitHub Packages

Objetivo: Generar un token de acceso personal con permiso de lectura de paquetes para que `npm ci` pueda descargar `@bopacorp/shared` durante la construcción.

- 1) Inicie sesión en GitHub con una cuenta que tenga acceso a `Bopacorp/bopacorp-shared`.
- 2) Navegue a *Settings* → *Developer settings* → *Personal access tokens* → *Tokens (classic)*.
- 3) Pulse *Generate new token (classic)* y asígnele un nombre identificable, por ejemplo `bopacorp-deploy`.
- 4) Establezca la caducidad. Para producción se recomienda 90 días con rotación programada, o bien una cuenta de servicio dedicada.
- 5) Marque **únicamente** el ámbito `read:packages`.
- 6) Copie el token generado. GitHub lo muestra una sola vez.

El token debe verificarse antes de intentar el despliegue, porque un token inválido no se manifiesta hasta la fase de construcción, varios minutos después:

```
export NPM_TOKEN=ghp_su_token_real

curl -s -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer $NPM_TOKEN" \
  https://npm.pkg.github.com/@bopacorp%2Fshared
```

Resultado esperado. El comando devuelve 200. Un 401 indica que el token es inválido o ha caducado; un 403 indica que carece del ámbito `read:packages`; un 404 suele significar que la cuenta no tiene acceso a la organización o que el nombre del paquete es distinto.

Nota. La alternativa moderna es un *fine-grained token* con permiso de sólo lectura sobre los paquetes de la organización, preferible por seguir el principio de mínimo privilegio.

4.6 Paso 6 — Obtener el proyecto de despliegue

Objetivo: Situar en el servidor la carpeta que contiene `deploy.sh`, `docker-compose.yml`, `.env.docker.example` y `Caddyfile.example`.

```
git clone <url-del-repo-de-despliegue> bopacorp-deploy
cd bopacorp-deploy
ls -la
```

Resultado esperado. La carpeta contiene, como mínimo, `deploy.sh`, `docker-compose.yml` y `.env.docker.example`.

Nota. El guion `deploy.sh` se ejecuta siempre desde su propio directorio y espera encontrar allí tanto `docker-compose.yml` como `.env`. Los tres repositorios de código se clonarán como subcarpetas de este directorio.

4.7 Paso 7 — Configurar el archivo `.env`

Objetivo: Definir todas las credenciales y URL públicas que consumen la API y los frontends.

```
cp .env.docker.example .env
nano .env
chmod 600 .env
```

La referencia completa de cada variable se encuentra en la Sección 5 y una plantilla lista para rellenar en el Anexo A. Los valores criptográficos deben generarse, nunca inventarse a mano:

```
openssl rand -base64 32 # JWT_SECRET
openssl rand -base64 32 # DOCUMENTS_ENCRYPTION_KEY
```

Antes de continuar, verifique la siguiente lista:

- ✓ `NPM_TOKEN` presente y validado en el Paso 5.
- ✓ `DATABASE_URL` (puerto 6543, con `pgbouncer=true`) y `DIRECT_URL` (puerto 5432) apuntan al proyecto Supabase correcto.
- ✓ `JWT_SECRET` tiene 32 caracteres o más y no es el valor de ejemplo.
- ✓ `API_PUBLIC_URL` usa el dominio real, con esquema `https://` y sin barra final.
- ✓ `CORS_ORIGIN` incluye los dominios del CRM y de la web, separados por comas y sin espacios.
- ✓ `VITE_API_URL` coincide exactamente con `API_PUBLIC_URL` seguido de `/api/v1`.
- ✓ Las credenciales de Supabase Storage y de Cloudflare R2 están completas.
- ✓ Los *buckets* `resumes`, `documents` e `images` existen ya en los proveedores.
- ✓ La IP pública del servidor está autorizada en *Database* → *Network Restrictions* de Supabase.

Resultado esperado. `ls -l .env` muestra permisos `-rw-----` y el archivo no contiene ningún valor de ejemplo sin sustituir.

Advertencia. El archivo `.env` concentra todos los secretos del sistema. Nunca debe subirse a un repositorio ni copiarse a carpetas sincronizadas. Verifique que `.gitignore` lo excluye.

4.8 Paso 8 — Ejecutar `deploy.sh`

Objetivo: Clonar o actualizar los tres repositorios, construir las imágenes y levantar los contenedores.

```
chmod +x deploy.sh
./deploy.sh
```

El guion opera con `set -e`, de modo que se detiene ante el primer error. Su ejecución consta de cuatro fases.

Tabla V
LÓGICA DE SINCRONIZACIÓN DE REPOSITORIOS (FASE 1)

Estado detectado	Acción del guion
La carpeta no existe	Clona el repositorio desde <code>github.com/Bopacorp/</code>
Existe y contiene <code>.git</code>	<code>git pull</code> para traer la última versión
Existe sin <code>.git</code>	Renombra a <code><repo>-backup-AAAAMMDD-HHMMSS</code> y clona de nuevo

Fase 1 — Sincronización de repositorios. Para cada uno de `bopacorp-api`, `bopacorp-web` y `bopacorp-crm`, el guion decide su acción según el estado de la carpeta, conforme a la Tabla V. El paquete `bopacorp-shared` no se clona.

Fase 2 — Construcción. Ejecuta `docker compose build`. Cada imagen parte de `node:22-alpine`, copia `package.json`, `package-lock.json` y `.npmrc`, ejecuta `npm ci` —momento en que se usa `NPM_TOKEN`— y copia después el resto del código.

Fase 3 — Arranque. Ejecuta `docker compose up -d`. La API arranca primero por la declaración `depends_on`, y los volúmenes montados activan la recarga en caliente.

Fase 4 — Mensaje de finalización.

Resultado esperado. El guion termina sin errores y su última línea confirma la finalización. La primera ejecución puede tardar entre 5 y 15 minutos.

Si falla. Anote el último mensaje impreso antes de la interrupción y consulte la Tabla X. Los tres fallos más frecuentes en este punto son un `NPM_TOKEN` inválido, la falta de memoria durante `npm ci` y la ausencia de credenciales de Git para repositorios privados.

Nota. El guion es idempotente en la práctica: puede reejecutarse cuantas veces sea necesario. Para reconstruir sin volver a sincronizar los repositorios basta con `docker compose build && docker compose up -d`.

4.9 Paso 9 — Verificar el despliegue local

Objetivo: Confirmar que los tres contenedores están en ejecución y responden en la interfaz local, antes de involucrar DNS y TLS.

```
docker compose ps
docker compose logs --tail=100 api
docker stats --no-stream
```

Los tres contenedores deben aparecer con estado Up. A continuación se comprueba que responden:

```
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:3001/
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:3002
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:3003
```

Resultado esperado. Los tres comandos devuelven un código HTTP en el rango 200–404 (cualquier respuesta HTTP demuestra que el proceso escucha). En la bitácora de la API debe aparecer el mensaje de arranque en el puerto interno 3000 y la conexión exitosa a la base de datos.

Si falla. Un código 000 indica que nada escucha en ese puerto: revise `docker compose logs` del servicio afectado. Si la API arranca y muere de inmediato, la causa habitual es un error en `DATABASE_URL` o una restricción de red en Supabase que bloquea la IP del servidor.

Nota. La ruta exacta del punto de verificación de salud de la API depende del proyecto. Si `/health` devuelve 404, consulte las rutas registradas en la bitácora de arranque.

4.10 Paso 10 — Configurar DNS

Objetivo: Hacer que los tres nombres de dominio resuelvan a la IP pública del servidor, condición previa para que Caddy pueda emitir los certificados.

En el proveedor de DNS deben crearse tres registros A apuntando a la IP pública del servidor, según la Tabla VI.

Tabla VI
REGISTROS DNS REQUERIDOS

Subdominio	Tipo	Valor
api-bopacorp	A	IP pública del servidor
bopacorp-crm	A	IP pública del servidor
bopacorp-web	A	IP pública del servidor

La propagación se verifica con:

```
dig +short api-bopacorp.tudominio.com
dig +short bopacorp-crm.tudominio.com
dig +short bopacorp-web.tudominio.com
```

Resultado esperado. Los tres comandos devuelven exactamente la IP pública del servidor.

Advertencia. Si el dominio está en Cloudflare, los registros deben permanecer en modo *DNS only* (nube gris) para que Caddy complete el desafío HTTP-01 de Let's Encrypt. Con el modo *Proxied* (nube naranja) el desafío falla, y la alternativa es usar un certificado de origen de Cloudflare o configurar el desafío DNS-01 con la API de Cloudflare.

Si falla. La propagación puede tardar desde unos minutos hasta varias horas según el TTL previo del registro. No avance al Paso 11 hasta que `dig` devuelva la IP correcta: intentar emitir certificados antes de tiempo consume la cuota de peticiones de Let's Encrypt.

4.11 Paso 11 — Configurar Caddy

Objetivo: Enrutar cada dominio al contenedor correspondiente y obtener los certificados TLS.

El proyecto incluye `Caddyfile.example` como plantilla mínima. La configuración recomendada añade compresión, cabeceras de seguridad y bitácoras:

```
{
    email operaciones@tudominio.com
}

api-bopacorp.tudominio.com {
    reverse_proxy 127.0.0.1:3001
    encode gzip
    header {
        Strict-Transport-Security "max-age=31536000; includeSubDomains"
        X-Content-Type-Options "nosniff"
        X-Frame-Options "SAMEORIGIN"
        Referrer-Policy "strict-origin-when-cross-origin"
    }
    log {
        output file /var/log/caddy/api.log
        format json
    }
}

bopacorp-crm.tudominio.com {
    reverse_proxy 127.0.0.1:3002
    encode gzip
    header Strict-Transport-Security "max-age=31536000; includeSubDomains"
    log { output file /var/log/caddy/crm.log }
}

bopacorp-web.tudominio.com {
    reverse_proxy 127.0.0.1:3003
    encode gzip
    header Strict-Transport-Security "max-age=31536000; includeSubDomains"
    log { output file /var/log/caddy/web.log }
}
```

El directorio de bitácoras debe existir y pertenecer al usuario del servicio, o Caddy no arrancará:

```

sudo mkdir -p /var/log/caddy
sudo chown caddy:caddy /var/log/caddy

sudo nano /etc/caddy/Caddyfile
caddy validate --config /etc/caddy/Caddyfile
sudo systemctl reload caddy
sudo systemctl status caddy --no-pager

```

Resultado esperado. `caddy validate` confirma la sintaxis, el servicio recarga sin errores y `journalctl -u caddy -f` muestra la obtención satisfactoria de los tres certificados en el plazo de unos segundos.

Si falla. Si el certificado no se emite, compruebe en este orden: que `dig` devuelve la IP correcta (Paso 10), que los puertos 80 y 443 están abiertos en el cortafuegos del servidor y en el grupo de seguridad del proveedor de nube, y finalmente la bitácora con `sudo journalctl -u caddy -n 100`.

Nota. Para entornos locales sin DNS público, Caddy puede emitir certificados con su autoridad interna usando nombres bajo `.localhost`, y `caddy trust` instala esa autoridad en el almacén del cliente.

4.12 Paso 12 — Validación final sobre HTTPS

Objetivo: Confirmar el funcionamiento extremo a extremo desde fuera del servidor.

```

curl -I https://api-bopacorp.tudominio.com
curl -I https://bopacorp-crm.tudominio.com
curl -I https://bopacorp-web.tudominio.com

```

Debe realizarse además una comprobación funcional desde un navegador: abrir el CRM, iniciar sesión y confirmar que las peticiones a la API se completan. Un frontend que carga pero no muestra datos indica casi siempre un desajuste entre `CORS_ORIGIN`, `API_PUBLIC_URL` y `VITE_API_URL`.

Resultado esperado. Las tres respuestas incluyen HTTP/2 200 (o una redirección legítima) y un certificado válido emitido por Let's Encrypt. En el navegador no debe haber errores de origen cruzado en la consola.

Si falla. Si el CRM carga pero no habla con la API, revise que `VITE_API_URL` se incrustó correctamente en la construcción. Recuerde que cambiar esa variable exige reconstruir la imagen: `docker compose up -d --build`.

4.13 Paso 13 — Endurecer el servidor

Objetivo: Reducir la superficie de exposición una vez que el sistema funciona.

Las medidas concretas se detallan en la Sección 9. Como mínimo deben aplicarse tres: restringir el cortafuegos a los puertos 22, 80 y 443; enlazar los puertos de Docker exclusivamente a la interfaz de bucle local; y deshabilitar la autenticación SSH por contraseña.

Resultado esperado. `sudo ss -tlnp` no muestra los puertos 3001–3003 escuchando en `0.0.0.0`, y un escaneo externo sólo encuentra abiertos 22, 80 y 443.

5 Referencia de Variables de Entorno

Todas las variables residen en un único archivo `.env` situado junto a `docker-compose.yml`. Las Tablas VII a IX describen su significado, agrupadas por función. La columna **Ob.** indica si la variable es obligatoria (✓) o si tiene un valor por defecto aceptable (○).

5.1 Precedencia de `VITE_API_URL`

En `docker-compose.yml`, el valor de `VITE_API_URL` para el CRM y la web se deriva de `${API_PUBLIC_URL:-...}/api/v1`. Es decir: si `API_PUBLIC_URL` está definida, se usa automáticamente; en caso contrario se aplica un valor por defecto codificado en el archivo, que apunta a un dominio distinto. Definir siempre `API_PUBLIC_URL` en `.env` evita que un despliegue quede silenciosamente apuntando al entorno equivocado.

Tabla VII
VARIABLES DE CONSTRUCCIÓN, BASE DE DATOS Y AUTENTICACIÓN

Variable	Ob.	Descripción
<i>Construcción de imágenes</i>		
NPM_TOKEN	✓	Token de GitHub Packages con ámbito <code>read:packages</code> . Se consume durante <code>npm ci</code>
<i>Base de datos (Supabase / PostgreSQL)</i>		
DATABASE_URL	✓	Conexión a través del agrupador, puerto 6543, con el parámetro <code>pgbouncer=true</code>
DIRECT_URL	✓	Conexión directa en el puerto 5432, empleada por las migraciones de Drizzle
<i>Autenticación</i>		
JWT_SECRET	✓	Secreto de firma de tokens; mínimo 32 caracteres, generado con <code>openssl rand -base64 32</code>
JWT_EXPIRES_IN	○	Vigencia del token de acceso. Compose fuerza 15m; en producción se sugiere 1d
JWT_REFRESH_EXPIRES_IN	○	Vigencia del token de refresco; valor recomendado 30d

Tabla VIII
VARIABLES DE URL PÚBLICAS, CORS Y BITÁCORAS

Variable	Ob.	Descripción
API_PUBLIC_URL	✓	URL pública del backend, p. ej. <code>https://api-bopacorp.tudominio.com</code> . La consumen ambos frontends
CORS_ORIGIN	✓	Orígenes autorizados, separados por comas y sin espacios
VITE_API_URL	✓	URL completa de la API con su prefijo de versión: <code>\${API_PUBLIC_URL}/api/v1</code>
LOG_LEVEL	○	<code>debug</code> , <code>info</code> , <code>warn</code> o <code>error</code>
LOG_PRETTY	○	<code>true</code> para salida legible en desarrollo; <code>false</code> (JSON) en producción

Tabla IX
VARIABLES DE ALMACENAMIENTO DE OBJETOS

Variable	Ob.	Descripción
<i>Supabase Storage — currículums</i>		
SUPABASE_URL	✓	URL base del proyecto
SUPABASE_S3_ENDPOINT	✓	Punto de acceso S3 del proyecto
SUPABASE_S3_ACCESS_KEY_ID	✓	Clave de acceso del almacenamiento
SUPABASE_S3_SECRET_ACCESS_KEY	✓	Secreto asociado a la clave
SUPABASE_STORAGE_BUCKET	✓	<i>Bucket</i> de currículums, p. ej. <code>resumes</code>
S3_REGION	○	Región declarada, p. ej. <code>us-east-1</code>
<i>Documentos cifrados</i>		
DOCUMENTS_STORAGE_BUCKET	✓	<i>Bucket</i> de documentos, p. ej. <code>documents</code>
DOCUMENTS_ENCRYPTION_KEY	✓	Clave de 32 bytes en base64
<i>Cloudflare R2 — imágenes públicas</i>		
R2_ACCOUNT_ID	✓	Identificador de cuenta de Cloudflare
R2_ACCESS_KEY_ID	✓	Clave de acceso de R2
R2_SECRET_ACCESS_KEY	✓	Secreto asociado
R2_BUCKET_NAME	✓	Nombre del <i>bucket</i> , p. ej. <code>images</code>
R2_PUBLIC_URL	✓	URL pública desde la que se sirven las imágenes

6 Configuración de Desarrollo Frente a Producción

La configuración provista está orientada al desarrollo y presenta cuatro diferencias sustanciales respecto de lo que exige un entorno productivo. Documentarlas explícitamente evita que se asuman garantías inexistentes.

6.1 Servidores de desarrollo en lugar de artefactos estáticos

Los contenedores `crm` y `web` ejecutan el servidor de desarrollo de Vite en el puerto 5173. Ese servidor no está diseñado para tráfico público: no aplica minificación agresiva ni división óptima de paquetes, mantiene el cliente de recarga en caliente activo y consume considerablemente más memoria. La configuración productiva correcta consiste en ejecutar `npm run build` en una etapa intermedia y servir la carpeta `dist` resultante mediante un servidor estático ligero, o directamente desde Caddy con la directiva `file_server`.

6.2 Dockerfile.dev en la API

La API se construye con `Dockerfile.dev`, que instala también las dependencias de desarrollo y ejecuta el proceso con recarga automática. Una imagen de producción debería emplear `npm ci --omit=dev`, una construcción en múltiples etapas y un usuario sin privilegios dentro del contenedor.

6.3 Volúmenes de código montados

El montaje de `./bopacorp-api/src` sobre `/app/src` implica que el contenido del contenedor no es el de la imagen, sino el del disco del anfitrión. En consecuencia, la imagen deja de ser el artefacto inmutable que define lo que se ejecuta, y un cambio accidental en el sistema de archivos del servidor altera el comportamiento en producción sin dejar rastro en el registro de imágenes. En producción estos volúmenes deben eliminarse.

6.4 Política de reinicio ausente

Sin la directiva `restart`, los contenedores no se recuperan tras un reinicio del anfitrión. Debe añadirse a los tres servicios:

```
services:
  api:
    restart: unless-stopped
  crm:
    restart: unless-stopped
  web:
    restart: unless-stopped
```

6.5 Recomendación

Mientras estas diferencias no se resuelvan, el despliegue descrito debe considerarse apto para entornos de integración, demostración y preproducción. Para tráfico real conviene abordarlas en el orden siguiente, de mayor a menor impacto: política de reinicio, eliminación de los volúmenes de código, construcción estática de los frontends y, por último, imagen de producción de la API.

7 Operación y Mantenimiento

7.1 Control del ciclo de vida

```
docker compose stop           # detener conservando contenedores
docker compose start          # reanudar
docker compose restart api    # reiniciar solo la API
docker compose down           # eliminar contenedores y redes
docker compose down -v        # ademas elimina volúmenes
```

7.2 Reconstrucción

```
docker compose build          # solo capas modificadas
docker compose build --no-cache # desde cero
docker compose up -d --build   # reconstruir y levantar
```

7.3 Inspección

```
docker compose logs -f --tail=200
docker compose logs -f api crm
docker compose exec api sh
docker compose exec api env | sort
```

7.4 Gestión del espacio en disco

```
docker system df          # uso por categoria
docker image prune -f     # imagenes huérfanas
docker builder prune -f   # cache de construccion
```

Advertencia. `docker system prune -a --volumes` elimina toda imagen no referenciada por un contenedor en ejecución. Es eficaz pero obliga a descargar y reconstruir todo en el siguiente despliegue; úselo sólo cuando el disco esté realmente comprometido.

8 Actualización y Reversión

8.1 Actualizar el código de la aplicación

```
cd bopacorp-deploy
./deploy.sh
```

El guion sincroniza los tres repositorios y reconstruye. Si hubo cambios en `package.json`, `npm ci` instalará las nuevas dependencias durante la construcción.

8.2 Actualizar la plataforma

```
sudo apt update
sudo apt upgrade -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo apt upgrade -y caddy && sudo systemctl reload caddy
sudo apt upgrade -y          # resto del sistema
sudo reboot                  # tras actualizar el nucleo
```

8.3 Procedimiento de reversión

Dado que el estado persistente es externo, revertir consiste en volver a una revisión anterior del código y reconstruir. Antes de una actualización de riesgo conviene anotar la revisión actual de cada repositorio:

```
for d in bopacorp-api bopacorp-crm bopacorp-web; do
  echo "$d $(git -C $d rev-parse --short HEAD)"
done > revisiones-previas.txt
```

Para revertir un servicio a la revisión anotada:

```
git -C bopacorp-api checkout <sha-anterior>
docker compose build api
docker compose up -d api
```

Advertencia. Si la versión revertida es anterior a una migración de base de datos ya aplicada, el esquema y el código dejarán de ser compatibles. Verifique siempre si la actualización incluyó migraciones antes de revertir, y utilice la recuperación a un punto en el tiempo de Supabase si es necesario deshacerlas.

9 Seguridad y Endurecimiento

9.1 Cortafuegos

Sólo tres puertos deben estar abiertos al exterior. Los puertos 3001–3003 no deben publicarse bajo ninguna circunstancia:

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp          # SSH
sudo ufw allow 80/tcp          # ACME HTTP-01
sudo ufw allow 443/tcp         # HTTPS
sudo ufw enable
sudo ufw status verbose
```

9.2 Docker y el cortafuegos: una precaución necesaria

Docker manipula directamente las cadenas de iptables e inserta sus reglas por delante de las de UFW. En consecuencia, un puerto publicado con ports: "3001:3000" queda accesible desde Internet **aunque UFW lo declare bloqueado**. Es un error de configuración habitual y de consecuencias graves, porque expone la API sin TLS ni control de origen.

La solución robusta es enlazar las publicaciones exclusivamente a la interfaz de bucle local, que es además todo lo que Caddy necesita:

```
services:
  api:
    ports:
      - "127.0.0.1:3001:3000"
  crm:
    ports:
      - "127.0.0.1:3002:5173"
  web:
    ports:
      - "127.0.0.1:3003:5173"
```

La verificación es inmediata: en `sudo ss -tlnp` las tres entradas deben mostrar `127.0.0.1:300x` y no `0.0.0.0:300x`.

9.3 Acceso SSH

```
# /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
```

```
sudo systemctl restart ssh
```

Advertencia. Confirme que su clave pública ya está instalada y que puede autenticarse con ella *antes* de deshabilitar la autenticación por contraseña. De lo contrario quedará bloqueado fuera del servidor y necesitará la consola de rescate del proveedor.

9.4 Gestión de secretos

- Mantener `chmod 600 .env` y propiedad del usuario que administra el despliegue.
- Rotar `JWT_SECRET` periódicamente, asumiendo que la rotación invalida todas las sesiones activas.
- No rotar `DOCUMENTS_ENCRYPTION_KEY` sin un plan de recifrado: los documentos existentes dejarían de poder descifrarse.
- Rotar `NPM_TOKEN` antes de su caducidad para no interrumpir los despliegues.

9.5 Actualizaciones automáticas de seguridad

```
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

10 Monitoreo y Observabilidad

10.1 Bitácoras

```
docker compose logs -f --tail=100 api      # aplicacion
sudo journalctl -u caddy -f                # proxy inverso
```

El nivel de detalle de la aplicación se controla con `LOG_LEVEL` y el formato con `LOG_PRETTY`. En producción se recomienda `info` y salida JSON, que es la forma consumible por un agregador externo.

10.2 Recursos

Tabla X
DIAGNÓSTICO DE FALLOS FRECUENTES (I): CONSTRUCCIÓN Y ARRANQUE

#	Síntoma	Causa probable	Acción correctiva
1	401 Unauthorized o 404 de npm al construir @bopacorp/shared	NPM_TOKEN inválido, caducado o sin el ámbito read:packages	Validar el token con la comprobación del Paso 5; regenerarlo con el ámbito correcto y actualizar .env
2	npm ci falla con error de resolución de dependencias	package-lock.json desactualizado o inconsistente con package.json	Ejecutar ./deploy.sh para sincronizar los repositorios y reintentar
3	La construcción se interrumpe con Killed o signal 9	Memoria insuficiente; el núcleo termina el proceso	Añadir o ampliar el espacio de intercambio (Paso 1); construir los servicios de uno en uno; o aumentar la RAM
4	Clonado con Permission denied (publickey)	Repositorios privados sin credenciales configuradas en el servidor	Instalar una clave SSH de despliegue o un token en la configuración de Git del servidor
5	docker: permission denied al ejecutar ./deploy.sh	El usuario no pertenece al grupo docker	sudo usermod -aG docker \$USER y volver a iniciar sesión
6	./deploy.sh: Permission denied	El guion carece del bit de ejecución	chmod +x deploy.sh
7	compose is not a docker command	Falta el complemento Compose V2	Instalar docker-compose-plugin (Paso 3)
8	port is already allocated al levantar	Los puertos 3001–3003 están ocupados por otro proceso	sudo lsof -i :3001 para identificarlo; detener el proceso o reasignar los puertos en Compose y en el Caddyfile

```
docker stats      # CPU, memoria y red por contenedor
df -h /           # espacio en disco
docker system df  # espacio consumido por Docker
```

10.3 Comprobaciones de salud

Docker Compose garantiza el orden de arranque mediante `depends_on`, pero no espera a que la API esté realmente lista. Declarar una comprobación de salud convierte esa dependencia en efectiva y permite que Docker reinicie un contenedor que ha dejado de responder:

```
services:
  api:
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:3000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 30s
```

Nota. Ajuste la ruta al punto de verificación que exponga realmente la API. Un `healthcheck` que apunta a una ruta inexistente marca el contenedor como no saludable de forma permanente.

11 Solución de Problemas

Las Tablas X y XI recogen los fallos observados con mayor frecuencia, ordenados aproximadamente según el momento del procedimiento en que se manifiestan.

11.1 Comandos de diagnóstico

```
# Construcción con salida detallada, guardada en archivo
docker compose build --progress=plain 2>&1 | tee build.log

# Variables efectivas dentro de un contenedor
docker compose exec api env | sort
```


Tabla XI
DIAGNÓSTICO DE FALLOS FRECUENTES (II): CONECTIVIDAD, DOMINIOS Y OPERACIÓN

#	Síntoma	Causa probable	Acción correctiva
9	La API no conecta con la base de datos	DATABASE_URL o DIRECT_URL incorrectas, o la IP del servidor está bloqueada en Supabase	Verificar ambas cadenas y las restricciones de red del proyecto; probar la conexión con <code>psql</code> desde el servidor
10	El CRM o la web cargan pero no obtienen datos	Desajuste entre VITE_API_URL, API_PUBLIC_URL y CORS_ORIGIN, o bloqueo por origen cruzado	Comparar las tres variables; confirmar que coinciden con los dominios servidos por Caddy; reconstruir para reincrustar VITE_*
11	Falla la subida de currículums, documentos o imágenes	Credenciales S3 o R2 incorrectas, o <i>buckets</i> inexistentes	Verificar las claves de Supabase Storage y R2 y comprobar que los tres <i>buckets</i> existen
12	Caddy no obtiene el certificado	El dominio no resuelve a la IP, los puertos 80/443 están cerrados, o Cloudflare está en modo <i>Proxied</i>	Comprobar con <code>dig</code> ; abrir los puertos en UFW y en el proveedor; pasar el registro a <i>DNS only</i>
13	La aplicación responde por IP pero no por dominio	Caddyfile mal configurado o cortafuegos	<code>caddy validate -config /etc/caddy/Caddyfile</code> , recargar y revisar <code>journalctl -u caddy</code>
14	Los cambios en <code>.env</code> no surten efecto	Los contenedores se crearon con los valores anteriores; las variables VITE_* se incrustan en tiempo de construcción	<code>docker compose up -d --build</code>
15	El disco se llena durante las construcciones	Acumulación de caché de construcción e imágenes huérfanas	<code>docker builder prune -f</code> e <code>docker image prune -f</code> ; verificar la rotación de bitácoras del Paso 3
16	<code>deploy.sh</code> se detiene a mitad sin mensaje claro	<code>set -e</code> aborta ante el primer comando fallido	Releer la última línea impresa antes de la interrupción y localizar el síntoma en estas tablas

```
# Resolución DNS
dig +short api-bopacorp.tudominio.com

# Puertos en escucha y a que interfaz estan enlazados
sudo ss -tlnp | grep -E '3001|3002|3003|:80|:443'

# Redes y conectividad entre contenedores
docker network ls
docker compose exec crm wget -qO- http://api:3000/ || true
```

12 Resumen Operativo

El siguiente bloque condensa el procedimiento completo para consulta rápida por parte de quien ya conoce el sistema.

```
# --- 1-4. PREPARAR EL SERVIDOR (una sola vez) ---
sudo apt update && sudo apt upgrade -y
sudo apt install -y git ca-certificates curl gnupg lsb-release
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER && newgrp docker

sudo apt install -y debian-keyring debian-archive-keyring \
  apt-transport-https
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/caddy-stable-archive-keyring.gpg
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' \
  | sudo tee /etc/apt/sources.list.d/caddy-stable.list
sudo apt update && sudo apt install -y caddy
sudo systemctl enable --now caddy

# --- 5-7. CREDENCIALES Y CONFIGURACION ---
cp .env.docker.example .env
nano .env # NPM_TOKEN, DATABASE_URL, JWT_SECRET, S3, R2...
chmod 600 .env
```

```
# --- 8-9. DESPLEGAR Y VERIFICAR ---
chmod +x deploy.sh && ./deploy.sh
docker compose ps
docker compose logs -f api
curl -s -o /dev/null -w '%{http_code}\n' http://localhost:3001/

# --- 10-11. DNS Y PROXY INVERSO ---
dig +short api-bopacorp.tudominio.com
sudo nano /etc/caddy/Caddyfile
caddy validate --config /etc/caddy/Caddyfile
sudo systemctl reload caddy

# --- 12. VALIDACION FINAL ---
curl -I https://api-bopacorp.tudominio.com
curl -I https://bopacorp-crm.tudominio.com
curl -I https://bopacorp-web.tudominio.com

# --- 13. ENDURECIMIENTO ---
sudo ufw default deny incoming && sudo ufw allow 22,80,443/tcp
sudo ufw enable

# --- MANTENIMIENTO ---
./deploy.sh                # actualizar y reconstruir
docker compose logs -f      # seguir bitacoras
docker compose restart api   # reiniciar un servicio
docker system df             # uso de disco
```

Anexo A

Plantilla Completa del Archivo .env

Los valores mostrados son marcadores de posición y deben sustituirse en su totalidad antes del despliegue.

```
# ===== CONSTRUCCION =====
NPM_TOKEN=ghp_reemplazar_por_token_real

# ===== BASE DE DATOS (Supabase) =====
DATABASE_URL=postgresql://usuario:clave@host:6543/postgres?pgbouncer=true
DIRECT_URL=postgresql://usuario:clave@host:5432/postgres

# ===== AUTENTICACION =====
JWT_SECRET=generar_con_openssl_rand_base64_32
JWT_EXPIRES_IN=1d
JWT_REFRESH_EXPIRES_IN=30d

# ===== BITACORAS =====
LOG_LEVEL=info
LOG_PRETTY=false

# ===== URLS PUBLICAS Y CORS =====
API_PUBLIC_URL=https://api-bopacorp.tudominio.com
CORS_ORIGIN=https://bopacorp-crm.tudominio.com,https://bopacorp-web.tudominio.com
VITE_API_URL=https://api-bopacorp.tudominio.com/api/v1

# ===== SUPABASE STORAGE (curriculums) =====
S3_REGION=us-east-1
SUPABASE_URL=https://proyecto.supabase.co
SUPABASE_S3_ENDPOINT=https://proyecto.supabase.co/storage/v1/s3
SUPABASE_S3_ACCESS_KEY_ID=reemplazar
SUPABASE_S3_SECRET_ACCESS_KEY=reemplazar
SUPABASE_STORAGE_BUCKET=resumes

# ===== DOCUMENTOS CIFRADOS =====
DOCUMENTS_STORAGE_BUCKET=documents
DOCUMENTS_ENCRYPTION_KEY=generar_con_openssl_rand_base64_32

# ===== CLOUDFLARE R2 (imagenes publicas) =====
R2_ACCOUNT_ID=reemplazar
R2_ACCESS_KEY_ID=reemplazar
R2_SECRET_ACCESS_KEY=reemplazar
R2_BUCKET_NAME=images
R2_PUBLIC_URL=https://images.tudominio.com
```

Anexo B

Documentación Oficial de Referencia

Documentación oficial de los componentes empleados: Docker Engine y Docker Compose (*docs.docker.com*), Caddy 2 (*caddyserver.com/docs*), Let's Encrypt y el protocolo ACME (*letsencrypt.org/docs*), GitHub Packages para el registro npm (*docs.github.com/packages*), Supabase — base de datos y almacenamiento (*supabase.com/docs*) y Cloudflare R2 (*developers.cloudflare.com/r2*).